

Essay Perbaikan Tugas Proyek 1
IF4070 Representasi Pengetahuan dan Penalaran
Ontologi dan Sistem Berbasis Pengetahuan



Disusun Oleh:

Albert

13522081

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

1. Penjelasan Perbaikan

Pada Tugas Proyek 1 IF4070, seluruh anggota kelompok berkontribusi dalam pembuatan kode Prolog, penyusunan ontologi, serta penulisan laporan. Pembagian kerja dilakukan secara bersama: pengumpulan data model BMW dan atributnya, perumusan fakta dan aturan Prolog, serta penyusunan laporan dilakukan bersama dan saling melengkapi kekurangan satu sama lain.

Saya, Albert (13522081), ingin menyampaikan klarifikasi terkait proses penulisan laporan. Pada versi laporan yang dikumpulkan sebelumnya, bagian BAB IV.2 (Keterlibatan logika deskripsi) dan BAB IV.3 (Keterbatasan ontologi dan sistem berbasis pengetahuan) sempat menggunakan bantuan Generative AI untuk memperbaiki pilihan kata dan merapikan alur antar kalimat. Penggunaan ini tidak ditujukan untuk membuat isi baru, melainkan untuk memperhalus penulisan agar terlihat lebih rapi.

Perbaikan ini akan difokuskan dengan memperbaiki penulisan di bab IV.2, IV.3 dan penambahan bab II.2.

2. Isi Perbaikan

Berikut isi perbaikan dari saya:

II.2. Proses Penyusunan Ontologi

Pada tahap awal, dilakukan identifikasi terhadap atribut-atribut yang secara umum digunakan untuk membedakan satu model kendaraan dengan model lainnya. Beberapa atribut yang sempat dipertimbangkan antara lain kapasitas mesin, tenaga maksimum (horsepower), torsi, serta harga. Namun, setelah dilakukan beberapa percobaan awal dalam memodelkan atribut-atribut tersebut ke dalam ontologi, ditemukan bahwa penggunaan atribut numerik yang terlalu detail justru menyulitkan proses klasifikasi. Atribut seperti horsepower dan torsi memerlukan penentuan batas nilai yang tidak selalu jelas, sehingga dapat menimbulkan kebingungan pada proses penalaran. Oleh karena itu, atribut numerik tersebut tidak digunakan sebagai bagian utama dari ontologi.

Sebagai alternatif, ontologi difokuskan pada atribut-atribut yang bersifat kategorikal dan relatif stabil, seperti series, body type, powertrain, drivetrain, feature, dan performance. Atribut-atribut ini umum digunakan dalam deskripsi resmi kendaraan BMW dan tidak terlalu bergantung pada perbedaan model. Selain itu, atribut kategorikal lebih sesuai untuk digunakan dalam penalaran berbasis aturan logika.

Setelah atribut utama ditentukan, penyusunan struktur kelas dan relasi dilakukan secara bertahap. Pada tahap ini, struktur ontologi dijaga agar tetap sederhana namun tetap mampu merepresentasikan perbedaan karakteristik antar model kendaraan. Beberapa atribut yang awalnya dipertimbangkan, tidak dimasukkan karena dapat memperluas domain secara signifikan tanpa memberikan kontribusi langsung terhadap tujuan klasifikasi.

IV.2. Keterlibatan Logika Deskripsi

Sistem klasifikasi pada proyek ini diimplementasikan menggunakan Prolog. Walaupun Prolog bukan reasoner Logika Deskripsi (Description Logic/DL), struktur pengetahuan yang digunakan tetap disusun dengan cara yang mirip dengan DL.

Berikut keterlibatan DL (Description Logic) yang diimplementasikan:

1. Pemisahan Konseptual TBox dan ABox

Sistem ini menerapkan pemisahan konsep antara TBox dan ABox seperti pada ontologi Logika Deskripsi. Bagian TBox direpresentasikan melalui aturan inferensi Prolog yang berperan sebagai definisi kelas, misalnya:

```
electric_car(M) :- has_powertrain(M, electric).
```

Aturan tersebut menjelaskan kondisi agar suatu model dapat dikategorikan sebagai mobil listrik. Sementara itu, bagian ABox direpresentasikan melalui fakta individu, misalnya:

```
has_powertrain(bmw_i7_xdrive60, electric).
```

Fakta tersebut menyatakan informasi untuk satu model tertentu, yaitu bahwa `bmw_i7_xdrive60` memiliki powertrain listrik.

Pemisahan ini membantu karena aturan pada TBox bisa digunakan kembali untuk banyak data ABox yang berbeda. Dengan begitu, ketika ada model baru yang ditambahkan (selama fakta dasarnya dimasukkan), aturan yang sama tetap bisa dipakai untuk melakukan klasifikasi. Hal ini membuat sistem lebih rapi dan lebih mudah dikembangkan.

2. Penggunaan Pola DL dalam Aturan Prolog

Beberapa bentuk aturan Prolog pada sistem ini dapat dipahami sebagai padanan konstruktor yang dipakai dalam Logika Deskripsi.

- Konjungsi (AND) direpresentasikan melalui koma (,) pada Prolog.

Misalnya, aturan:

```
offroad_capable_suv(M)      :-      has_body_type(M,
suv), has_feature(M, xdrive).
```

Artinya berarti sebuah kendaraan masuk kategori offroad capable SUV jika merupakan SUV dan memiliki fitur xDrive.

- Disjungsi (OR) direpresentasikan melalui tanda titik koma (;).

Contohnya:

```
high_performance_car(M)    :-      has_performance(M,
m_series) ; has_performance(M, m_performance).
```

Artinya sebuah mobil dianggap berperforma tinggi jika termasuk M Series atau M Performance.

- Kuantifikasi eksistensial pada DL dapat dipahami secara implisit melalui relasi seperti `has_X(M, Y)`, yang menunjukkan bahwa model M memiliki suatu atribut bernilai Y. Dengan pola tersebut, aturan Prolog tetap sejalan dengan cara kerja Logika Deskripsi.

3. Subsumsi dan Klasifikasi Implisit

Sistem ini melakukan klasifikasi secara implisit: ketika suatu model memenuhi kondisi definisi kelas, sistem akan menyimpulkan bahwa model tersebut termasuk dalam kelas tersebut. Sebagai contoh, karena `bmw_m3g80` memiliki `performance = m_series`, dan kategori tersebut termasuk syarat untuk `high_performance_car`, maka sistem akan menyimpulkan bahwa `bmw_m3g80` adalah mobil berperforma tinggi.

Walaupun hubungan hierarki kelas tidak ditulis secara eksplisit dalam bentuk “subclass of”, hasil klasifikasi tetap mengikuti definisi kelas yang dibuat. Dengan cara ini, sistem tetap bisa melakukan pengelompokan berdasarkan konsep, seperti yang menjadi tujuan utama DL.

4. Keterbatasan Representasi DL dalam Prolog

Walaupun prinsip DL diikuti, Prolog tetap memiliki beberapa keterbatasan jika dibandingkan dengan reasoner DL formal. Misalnya, dalam Prolog tidak ada cara langsung untuk menyatakan hirarki kelas dan membuat sistem secara otomatis memahami hubungan tersebut. Selain itu, pembatasan kardinalitas seperti “tepat satu powertrain” juga tidak bisa dipaksakan otomatis tanpa aturan tambahan. Beberapa bentuk negasi atau disjungsi yang lebih kompleks juga tidak selalu nyaman diterapkan dalam Prolog standar. Namun, dalam konteks domain proyek yang dibatasi dan data yang terkontrol, keterbatasan ini masih bisa dikelola dengan penulisan aturan yang jelas serta asumsi domain yang konsisten.

IV.3 Keterbatasan ontologi dan sistem berbasis pengetahuan

Berikut adalah beberapa keterbatasan sistem, baik dari sisi representasi ontologi maupun dari sisi implementasi menggunakan Prolog.

1. Kurangnya Dukungan untuk Hierarki Kelas Eksplisit

Sistem saat ini belum membedakan secara formal antara subkelas dan kelas yang berdiri sendiri. Dalam Logika Deskripsi, hubungan subkelas dapat dinyatakan secara langsung, sehingga suatu kelas bisa dinyatakan sebagai bagian dari kelas lain. Namun pada implementasi Prolog yang digunakan, nilai

seperti *m_performance* dan *m_series* hanya diperlakukan sebagai kategori, bukan sebagai kelas yang memiliki hubungan hierarki.

Akibatnya, sistem tidak bisa secara otomatis memahami hubungan bertingkat antar kelas. Jika ingin membuat hubungan seperti itu, aturan tambahan harus ditulis secara manual. Hal ini membuat aturan menjadi lebih banyak dan kurang efisien jika sistem diperluas.

2. Tidak Ada Penanganan Konflik atau Inkonsistensi Secara Otomatis

Prolog tidak menyediakan pemeriksaan otomatis terhadap konsistensi data. Jika terdapat fakta yang saling bertentangan, misalnya suatu model diberi dua jenis *powertrain* yang berbeda, sistem tetap akan menerima keduanya tanpa peringatan. Hal ini berarti konsistensi basis pengetahuan sangat bergantung pada ketelitian saat memasukkan fakta. Tanpa pengawasan manual atau aturan tambahan, inkonsistensi bisa terjadi tanpa terdeteksi.

3. Ketergantungan pada Enumerasi Nilai (Closed-World Assumption)

Sistem bekerja dengan asumsi bahwa semua nilai atribut sudah diketahui dan ditentukan sebelumnya. Pendekatan ini cocok untuk domain yang ruang lingkungannya sudah ditentukan dan relatif stabil. Namun, jika muncul kategori baru, misalnya jenis *powertrain* baru, nilai tersebut harus ditambahkan secara manual agar dapat dikenali sistem. Dengan kata lain, sistem tidak secara otomatis menyesuaikan diri terhadap perubahan kategori yang belum didefinisikan.

4. Kurangnya Representasi untuk Properti Fungsional

Beberapa atribut seharusnya hanya memiliki satu nilai untuk setiap model, misalnya *powertrain* atau *drivetrain*. Namun, Prolog tidak secara otomatis membatasi hal tersebut. Jika tidak ditambahkan aturan khusus, sebuah model dapat saja memiliki lebih dari satu nilai pada atribut yang sama.

Untuk mencegah kondisi ini, perlu ditambahkan aturan pembatas (constraint) secara eksplisit. Sebagai contoh, inkonsistensi dapat dideteksi dengan aturan berikut:

```
has_powertrain(M, P1), has_powertrain(M, P2), P1 \= P2.
```

Aturan tersebut bertujuan untuk memastikan bahwa satu model tidak memiliki dua jenis *powertrain* yang berbeda. Tanpa aturan semacam ini, sistem tidak akan mendeteksi pelanggaran terhadap atribut yang seharusnya bernilai

tunggal.

5. Keterbatasan dalam Ekspresi Aturan Kompleks

Untuk klasifikasi yang lebih lanjut atau kombinasi fitur yang cukup banyak (misalnya mobil listrik mewah dengan ADAS Pro dan panoramic roof), sistem tetap memerlukan penulisan aturan secara manual untuk setiap kombinasi kondisi. Sistem ini belum mendukung pembentukan klasifikasi secara langsung dari gabungan kondisi pada saat query dijalankan. Artinya, klasifikasi yang lebih kompleks tidak bisa dibuat secara otomatis hanya dengan menyusun ekspresi kelas seperti pada lingkungan DL formal. Hal ini membuat fleksibilitas sistem terbatas pada aturan yang memang sudah didefinisikan sebelumnya.

6. Tidak Ada Dukungan untuk Waktu atau Evolusi Model

Ontologi yang digunakan bersifat statis dan belum memasukkan aspek waktu. Sistem tidak merepresentasikan tahun produksi atau perbedaan generasi model, misalnya perbedaan antara BMW 3 Series F30 dan G20. Padahal, beberapa atribut teknis seperti jenis drivetrain pada 1 Series dapat berubah antar generasi. Karena tidak ada dimensi waktu, klasifikasi hanya berlaku sesuai data yang sudah dimasukkan. Jika diterapkan pada model dari periode yang berbeda tanpa penyesuaian data, hasilnya bisa kurang tepat.

7. Ketergantungan pada Pengetahuan Eksplisit

Sistem berbasis Prolog ini sepenuhnya bergantung pada fakta dan aturan yang sudah dimasukkan sebelumnya. Sistem tidak memiliki kemampuan untuk menyimpulkan pola umum di luar aturan yang telah ditulis. Sebagai contoh, meskipun semua SUV BMW dalam data yang ada menggunakan xDrive (AWD), sistem tetap tidak akan menyimpulkan bahwa SUV baru pasti menggunakan AWD, kecuali fakta `has_drivetrain(ModelBaru, awd)` dimasukkan secara eksplisit. Oleh karena itu, pembaruan pengetahuan tetap harus dilakukan secara manual agar sistem tetap akurat.